

小芽：宠物对战游戏里的 LLM 教练

设计与实现说明

1 我怎么理解这道题，以及先砍掉了什么

题目给的场景是宠物对战游戏：玩家养一批精灵，打 PVP，配阵容，日常培养。要交付的是一个 LLM 教练，具备军师、陪练、老师三种角色感，至少完整闭环其中一项，其余可以只做设计。参考方向列了 Agentic RL、RAG、ReAct、Memory Stream / Reflection，产品参考是和平精英的”小田”和王者荣耀的”灵宝”。

拿到题目我先做的不是写代码，而是划边界。三件事我判断为本次不做，理由写在下面，因为不做的决定和做的决定一样需要交代。

不做三渲二的美术和完整游戏内容。题目要的是教练，游戏是产生局面的环境。我把宠物控制在 12 只、5 个关卡，够产生”抢先收尾还是留能量””换宠承伤还是原地治疗”这类真实取舍就停手。继续堆宠物不会让教练更聪明，只会让平衡工作吃掉全部时间。

不做 LLM 权重训练。本机是 arm64、没有可用的 CUDA 后端，也没有云上 GPU 预算。我做了两个能在这台机器上真跑完的训练实验（后面第 4 节和第 6 节会讲它们各自是什么、不是什么），但没有把它们包装成”训练了模型”。

不做联网 PVP。本地对战我做成了同一台设备上分屏轮流操作。它离真实竞技还有距离，这一点我在第 5 节里明说了，没有含糊过去。

三个角色里，军师和老师我做到了完整闭环，陪练只做到设计。这个划分是刻意的，也符合题目”其它可仅做设计”的说法。但我在第 3 节会把话说清楚，避免读的人以为三样都做完了。

2 怎么看这个东西

本地跑起来：

```
cd pet-coach-game
npm start          # Node.js 20+
```

浏览器打开 <http://127.0.0.1:8765/>。想看模型能力需要先在 `/connect.html` 里录入 DeepSeek 密钥，密钥只留在后端进程内存里；不录也能玩，规则建议、复盘、培养建议和小测都不依赖模型。

进页面是营地，顶部有两个入口：**训练**和**对局**。训练按关卡打固定对手，教练全程给建议；对局里对手从全部 12 只里自动配队、等级按你的队伍适配，教练照常工作。对局还有第二种对手：真人同机，两个人分屏各选各的，两边各有一条教练。

想快速看到教练在做什么，有两处最直接：一是战斗页底部左右两侧的教练条，各自只分析自己那一侧的局面；二是打完一局后自动出现的整局复盘，它会指出具体是哪一回合、当时有哪些别的选择。

3 三个角色各自做到哪一步

3.1 军师：完整闭环

军师是这次的重心。它的工作方式是把”算”和”说”分开：所有伤害、行动顺序、合法性由游戏引擎枚举，模型不参与计算，只负责把比较结果讲成玩家能理解的取舍。

具体做法是，玩家面对一个局面时，引擎枚举双方所有合法行动，对每一种组合跑一遍共享结算器，得到平均收益和最坏分支。同时顺序（双方同速）会分别按两种可能各算一次，这样搜索不会偷偷利用随机数。模型拿到的证据里包含这些数字、每张战术卡的适用条件和反例，它要做的只是解释。

举例说明这个分工为什么必要。同一个残血目标，火花直接伤害 52，追猎 75，两个都够击倒，都耗 2 豆。如果只看面板数字，很自然会告诉玩家”选追猎”。但这个判断是错的：对方可能吃药、可能换宠、可能防御，而追猎的额外伤害来自”目标已灼烧”这个前提，前提一变结论就变。这类判断交给模型去猜是猜不准的，交给枚举器是确定的。

3.2 老师：完整闭环

复盘分成整局和单回合两层。整局先给行动分布和回合数，再挑几个造成减员或血量大幅变化的回合；单回合用回合前的局面做条件比较，比如”当时目标 78 血，火花不防御打 24、防御打 8，单次攻击不足以收尾”。

关键的一条是：复盘一律使用回合开始时的公开状态，不使用对手实际打出的那一招。理由很直白，如果拿事后结果当依据，任何一次失败都能被解释成”你当时选错了”，而这既不公平也没法教人。这条规则有测试保证，改对手的实际行动不会改变复盘结论。

练习跟着复盘走。玩家在某个回合做了选择，教练按这个选择对应的知识点出一道题（灼烧追击、防御节奏、换宠承伤、先制与速度四类）。出题、等待作答、判题、取消这四个状态由程序控制，模型不能提前报答案，也不能把题目改写成别的。这道题和玩家刚才的真实选择挂钩，不是随机题库。

3.3 陪练：只做了设计

这是我这次最薄的一块，我把现状说清楚。

已经实现的只有：玩家表达挫败时的克制回应、追问时接住上文、以及基于真实对战历史的开场白（”记得上一场是在青芽草地”）。代码量很小。

没有实现的是”有情绪”这件事本身。我写了设计文档，里面做了一套情绪状态和对应的语气档位，但代码里没有。所以如果问”陪练有没有情绪”，答案是：设计上有，

实现上没有。

为什么没做？两个原因。一是题目允许非主角角色只做设计，我把时间投给了军师和老师；二是我对“给游戏助手加情绪”这件事本身有保留。项目调研过的灵宝资料里有一条我认同的判断：情绪陪伴很容易变成油腻，玩家不打开聊天不代表需要被安慰。所以我的设计文档里给它加了比一般助手更严的约束——允许沉默、不主动提问、不空泛安慰、不因为一次失败就弹话。

4 难点，以及我是怎么改的

这一节按“我本来以为是什么样 → 哪里不对 → 改了什么 → 现在是什么数”来写。选的都是真卡住过我的地方。

4.1 玩家不打开聊天，就等于没有教练

第一版做出来我自己试了两局，发现一件事：小芽的所有能力都在聊天面板里，而我全程没打开过那个面板。一个要玩家主动召唤才存在的助手，在真实游玩里等于不存在。

改法是把主入口从聊天框挪到游戏事件上。现在的规则是：开局和关键风险变化时，在战斗页底部出现一行短提示，解释收在二级入口里按需展开；玩家出招之后，旧提示立刻隐藏，等新局面重新评估。聊天框降级成补充入口。

这条改动有个副作用值得说：提示一多就变成噪音。所以门控比内容更难做，见后面第五个难点。

4.2 数字不能让模型算

我一开始让模型看局面然后给建议，试出来的回答里数字是错的。不是偶尔错，是稳定地错——它会把不存在的技能名编出来，会把伤害算错，会在两个都能击倒的技能里挑面板数字大的那个。

后来把职责切开：引擎枚举双方所有合法行动，对每种组合同跑一遍共享结算器，得到平均收益和最坏分支。模型拿到的是一份事实包，里面只有引擎算出来的数字和每张战术卡的反例，它负责把这些讲成取舍。

有个例子能说明为什么这个分工是必须的。同一个残血目标，火花直接伤害 52，追猎 75，两个都够击倒，都耗 2 豆。只看面板会选追猎。但追猎的额外伤害来自“目标已灼烧”这个前提，对方吃药、换宠、防御都会让前提失效。这种判断交给模型是猜，交给枚举是算。

4.3 检索没有赢过关键词

我做过一个假设：把检索从关键词换成“关键词 + 多语言向量”的混合方案，口语化提问的命中率会提高。为了验证它，我写了一个对照实验，同一批查询分别跑无检索、纯关键词、纯向量、混合四档。

结果是混合方案没有赢。测试集 14 条正例上，关键词命中 11 条 ($\text{Hit}@3 = 0.786$)，混合也是 11 条 (0.714)，差异来自排序而不是命中的数量。负例拒答上两者更接近，关键词拒掉 1 条，纯向量拒掉 2 条。

这里我差点犯一个错。看到混合没赢，我的第一反应是“检索没用，RAG 是伪需求”。但把四档拆开看就发现不是这么回事：纯向量在负例拒答上比关键词好，混合的 MRR 从 0.667 升到 0.750，排序确实更好了。问题是这些改善都发生在 14 条正例、2 条负例的样本上。一条查询的差别就能让 $\text{Hit}@3$ 动 7 个百分点。

所以我没有拿这个结果去证明任何事。既没有换成向量检索，也没有宣称 RAG 无效，而是把它记成“这次测量没测出差别”，并把关键词保留为默认。后来读 BEIR 那篇才知道这类现象在检索领域很常见：BM25 在跨域零样本场景下本来就是很难打败的基线，稠密和稀疏检索模型在原论文里的表述是“often underperform”。

真正的教训是我一开始把评测集做得太小。20 条查询、4 条开发、16 条测试，这个规模连自己的结论都支撑不住。

4.4 模型会说过期的话

这个问题的表现是：玩家已经出招了，模型那边的解释才回来，界面上出现一句针对已经过去的局面的建议。

我试过在提示词里写“如果局面已经变化就不要回答”，没用。模型看不到自己那句回答会在什么时候被显示出来，这个判断只能由程序做。

改法是把每个请求绑定到具体的局面版本：请求发出时记下对局 ID、回合号、规则版本和上下文 epoch，返回时逐项比对。任何一项变了就丢弃结果，不显示、也不播报。这条对语音同样适用。

后来发现还有一个更隐蔽的变体：不是结果过期，是我自己的版本号没同步。服务端校验上下文模式的白名单是 `camp / pve / pvp-live`，我加本地对战时客户端开始发 `pvp-local`，但白名单没跟着改。结果是本地对战里的模型请求全部被 400 拒绝、静默回退成本地规则。规则建议照常工作，界面没有任何异常提示，这个模式从头到尾没有产生过一次模型解释。它是靠逐条核对才发现的，现在有一个回归测试遍历客户端可能发送的每一种模式。

4.5 模型会编道具名，也会把取消的行动说成命中

有两类错误是数字校验抓不到的。

第一类是道具名。这个游戏里只有回复药、净化药、能量果三样。模型会把净化药说成“解药”、能量果说成“以太”。数字全对，引用全对，只有名字是错的——因为它没有数字可校验。

第二类是因果。宠物倒下时，它这一回合已经排好的行动会被取消。模型会把这次取消描述成“命中了”。同样没有数字可比。

这两类都靠加检查解决：一是维护一份别名黑名单，命中就判不合格；二是读回

合事件，如果某个行动被记为取消，那么正文里就不能出现这个行动造成了伤害。判不合格之后走既有的降级路径，显示规则结论而不是错误描述。

4.6 什么时候不说话，比说什么更难

提示做出来之后，问题从”说得对不对”变成”该不该说”。我给的门控是逐项判断的：玩家是否在前台、是否已经出招、是否已经忽略过、这个回合是否已经提示过、上次提示距今多久、当前风险是否构成可行动的风险。安静档和”本局不再提醒”永远优先，不被模型推断覆盖。

还有一条是我从一篇教育数据挖掘的论文里学到的。那篇讲 assistance dilemma，作者报告说他们早期版本的预测器没有把”学生用了提示”这件事算进去，结果因为提示本身会把人推向高效步骤，用过的学生反而被判定成”不需要帮助”。他们的修正是把两者分开。

小芽里对应的是同一件事：判断玩家是否掌握某个知识点时，只用他没有被提示过的时候做出的选择。代码里就是一个过滤条件 `!e.prompted`。我是先按直觉这么设计的，后来才发现这个踩坑在文献里有记录。

4.7 对局里的对手，我没有交给模型

题目场景是 PVP，我一开始的想法是让模型来当对手，理由是”这样显得 AI 更强”。查了资料之后放弃了这个想法。

放弃的理由不是我懒，是证据方向相反。PokéLLMon 是一个纯靠语言模型打宝可梦的工作，阶梯赛胜率 49%。PokéChamp 换成”LLM 辅助 minimax 搜索”之后，对最强规则型 bot 胜率是 84%，而且不需要任何额外训练。Metamon 进一步用离线强化学习加 Transformer，论文里说它的表现超过了 LLM 智能体和强启发式搜索引擎。也就是说在这个游戏类型里，决定胜负的是搜索和训练，不是语言能力。

所以对手出招我用引擎的启发式：轻松档随机、标准档优先伤害和治疗、挑战档枚举双方合法行动并比较一回合结果。模型完全不参与。这样做的另一个好处是本地出招零延迟、零成本，也不会返回非法行动。

顺带说一个由此产生的设计问题。做了本地对战之后我发现，”电脑对手”和”训练模式”在玩法上其实是同一个东西——同一套 AI、同一套结算，唯一的区别是教练闭不闭嘴。这个区别是真的，但我原来把它藏在名字后面，第一次给用户看的时候被直接问了”这两个有啥区别”。现在两个入口的名字直接写明差别：训练是”按关卡挑战”，对局是”对手自由配队”。

4.8 教练在 PVP 里该不该说话，我改错了两次

这一条是我这次最值得记的。

题目对军师的描述是”在对战/阵容场景给出决策建议”，而场景设定写的是 PVP 对战。也就是说军师本来就该在对战里工作。

我第一版的做法是：只要是对局模式，教练一律不介入。理由是”两人共用一块

屏，给一方提示就是不公平”。这个理由听起来成立，还把”公平性设计”写进了文档当成亮点。

问题是这样等于把主功能在题目指定的场景里禁用了。改法是把判据从”是不是对局模式”换成”有没有第三方受影响”：对手是真人时闭麦，对手是电脑时照常工作。

改完之后又有人指出这个版本也不自洽：电脑对手存在的意义就是模拟真人，那它和真人局的行为就该一致，否则这个模拟本身就是假的。这句话是对的。最终改成本地对战一律允许教练，只保留线上竞技的闭麦开关。真正需要守住的底线不是”整局不说话”，而是”不读取对方待执行的动作”——那条项目一直没破，也有测试。

前后改了三次，前两次都是我把自己的推论当成了结论。

4.9 ”公平”这件事最后是靠对称解决的，不是靠闭麦

接着上一条。既然本地对战两边都能用教练，那公平性就不需要靠禁用某个功能来实现了。

具体做法是双方各有一条教练条，跑同一套引擎、同一套规则，各自只分析自己那一侧。对面用的是把 player 和 enemy 对调的镜像上下文，所以它看到的是对方的局面，也读不到任何隐藏信息——待执行的动作本来就不在上下文里。

实测两条教练在同一回合给的结论是不同的，而且各自正确：一方建议”火花”，另一方建议”换上晶角犀”。

4.10 对手要像真人，就得看不见我在干什么

本地对战的对手如果是 AI，它和真人的差别会体现在一个地方：真人不可能知道你选了什么。

我把这件事做成顺序上的保证，而不是提示词里的约束。每回合开始时，先让对手独立做决定——它读的是回合前的公开局面——然后才开放我方的操作面板。它决定的时刻早于我的选择，所以不可能参考我的选择。对方面板会直接写明”已独立出招（看不到你的选择）”，按钮也是禁用的，不是点了没反应。

4.11 分屏之后，两个面板必须一模一样

第一版分屏我只给我方面板做了标签页（技能/换宠/物品/逃跑），对方面板是一个平铺的列表。自己看着别扭，被指出之后才发现更实际的问题：右边有标题行、左边没有，导致右侧整块内容比左侧低了大约 30 像素，标签、按钮、教练条全部错位。

改成两侧走同一个渲染函数，结构完全一致，只有”逃跑”和”认输”的区别——引擎里 escape 的语义属于本地撤退，不能给对手用。中间加了一道竖线分隔。改完实测两侧的标题、标签行、技能区、教练条纵坐标完全对齐。

5 我做错的

上面那节讲的多数是”发现问题然后改对”，这一节讲三件我判断错了、并且错得有一段时间没意识到的事。

5.1 语音：连续三次判断错方向

玩家反馈语音有两个问题：播报完有一声爆鸣，而且第一次开启是普通话、后面变成粤语。

我的第一次判断是”系统里有英文语音在读中文”。这个判断错了。用 `say -v '??'` 把系统语音列出来才看到，Eddy、Flo、Grandma 这些名字确实各有真正的中文版本，示例文本就是”你好！我叫 Eddy”，不是英文语音冒充中文。

第二次我改成语音选择逻辑，优先精确匹配 `zh-CN`。也错了，实测两个中文语音都还是粤语。

第三次我把爆鸣归因于 `speechSynthesis.cancel()` 之后没有等待，加了 300 毫秒的间隔。还是不行。到这一步我才意识到前两次都是在没有证据的情况下改代码——我看到一个现象就猜一个原因，改完让使用者去试，试完再猜下一个。

真正定位问题是从使用者提供的一条信息开始的：”一开始是对的，取消一次再开启就又是粤语加爆鸣。”中间唯一发生的事就是 `cancel()`。也就是说这个操作会持久破坏后续播报的声音选择，只有刷新页面能恢复；而 `cancel()` 在代码里有九个调用点，包括每次出招，所以实际对战中每回合都会触发一次。这解释了”只有第一次是对的”。

改法是把 `cancel()` 全部删掉，代价是已经开口的那一句不能中途掐断。但改到这一步，验证的成本已经明显高于这个功能的收益了——音频我没有办法自己验证，无头浏览器没有声卡，每一次判断都要占用使用者的时间。所以我最后的选择是**整体停用语音**，代码留在开关后面，文档里如实写明停用原因，不再对外宣称具备这个能力。

这件事的教训不是”要小心”，是**在没有独立验证手段的情况下，不要让使用者反复替我做实验**。第一次失败之后就该做的是把变量拆开做成一个可以一次问清的东西，而不是连续猜。

5.2 一段 CSS 从来没有被写进文件

营地页的配招卡片排版很乱，名称、数值和说明挤在同一行。我看着截图以为是设计问题，准备重做布局。

实际原因是那段 CSS 根本没写进样式表。我当时把追加 CSS 的命令用 `&&` 串在了 `node --check` 后面，而那一刻 `app.js` 正好因为我刚打错一个括号而报错，于是 `cat >> style.css` 被跳过了。后面那句 `echo` 照样打印成功，我就以为写进去了。

后果是技能卡退回成 `inline-block`，所有文字连成一行。我按”设计问题”去改，改了也没用。

修法很简单，但过程值得记：**用命令的退出码去判断上一步是否成功，而不是

用它的输出 `**`。后面我改成逐个 `grep` 确认每个选择器真的落盘了。

5.3 两次重启后端，两次把密钥弄丢

密钥按设计只保存在后端进程的内存里，不落盘。我为了加载服务端改动重启了两次，两次都让使用者重新录入。

第一次还可以说是不知道，第二次是明知道的——我甚至提前写了提醒。真正的问题是我没有把”重启会清掉密钥”当成一个需要设计的东西，而是当成一个每次都要打招呼的麻烦。

后来加的改动让这件事好一些：客户端遇到 403 会自动重建会话并重试一次，所以重启之后使用者不会再看到一个莫名其妙的”模型不可用”；错误信息也不再一律显示”暂不可用”，而是把真实原因（会话失效、鉴权失败、超时）带出来。这些都是补丁，根因是这个项目从头到尾没有一个稳定的运行方式。

5.4 一个用文件名骗过我的数据文件

有一次我看到 `reports/live-model-eval.json` 存在，就以为真实模型评测跑完了。实际上那份文件只有两行，是更早一次局部验证留下的；完整的那次运行还没有做完。

这件事提醒我，交付物里文件名和内容要能对应上，中间产物不要用最终产物一样的名字。后来我在整理清单时对每一条都去看实际数据，而不是看文件名。

6 数字

下面每个数都是我自己跑出来的，附了产生它的命令或文件。带区间的表示不同运行之间的波动。

6.1 规模

项目	值
宠物 / 关卡 / 难度	12 / 5 / 3
技能 / 道具	23 / 3（回复药、净化药、能量果）
知识卡	90 张（49 张自写战术卡 + 41 条从引擎导出）
自动测试	131 项通过（ <code>npm test</code> ）
上下文窗口	1,000,000 tokens（DeepSeek 官方文档， <code>deepseek-flash</code> ）
工作预算	200,000 tokens

6.2 检索对照

同一个查询集跑四档，测试集 16 条，其中正例 14、负例 2。

读法：混合方案的排序质量比关键词好（MRR 0.667 到 0.750），但命中数量没有

策略	Hit@3	MRR	负例拒答
无检索	0.000	0.000	0/2
关键词（现默认）	0.786	0.667	1/2
概念扩展混合	0.714	0.750	1/2
纯语义向量	0.643	0.536	2/2

增加；纯向量在负例拒答上最好。这些差异都建立在 14 条正例上，一条查询的变动就能让 Hit@3 动约 7 个百分点，因此我没有据此更换默认策略。

6.3 两个训练实验

两个都不是”训练了 DeepSeek”，边界写在表里。

	干预策略实验	工具选择实验
学的是什么	教练何时开口（沉默 / 短提示）	该调哪个工具
方法	表格 Q-learning	REINFORCE + KL 正则
规模	3 个种子 × 6000 局	冻结骨干，更新 1152 个参数
数据	5 类模拟玩家	24 训练 / 8 验证 / 16 测试 (英文二选一)
结果	回报 1.116 (规则策略 -1.113)	测试集 0.5 → 0.9375，三个种子一致

两个实验都有一处必须说明的地方。干预策略的回报更高，但它的 ** 帮助次数反而更少 ** (5.23 对 7.23)，增量帮助也更低 (2.20 对 2.43) ——也就是说它赢在打扰成本，不是赢在教得更好，这两件事不能混着讲。工具选择实验的 16 条测试全部是英文、二选一、共 24 条训练样本，它证明的是”参数确实更新了并且学到了这个窄任务”，不是”模型会选工具”。

6.4 真实模型调用

2026-09-17 采集，五条：补位建议、败局分析、连续追问、自动总结、天气规则。

指标	值
延迟	3134–3722 ms
输入 tokens	1767–2420（只含最终生成，未合计规划调用）
工具轨迹	每条 1–2 次，如天气那条是 search_rules → read_state
已验证的行为	PVP 前置拒绝在 3 ms 内本地返回，无上游调用
已发现的缺陷	把净化药说成”解药”、能量果说成”以太”；数字校验没有拦住

最后一行是这次采集最有价值的产出。它直接导致了后来加的名称校验与因果校验。

6.5 一次真实的失效

服务端校验上下文模式的白名单只有 `camp / pve / pvp-live` 三个值。加本地对战时客户端开始发 `pvp-local`，白名单没跟着改。结果是本地对战里的模型请求全部返回 400「教练上下文无效」，客户端静默降级为本地规则。规则建议照常工作，界面上没有任何异常，这个模式从头到尾没有产生过一次模型解释。

发现方式是逐条核对，不是测试报警——因为没有任何测试覆盖”客户端可能发送的每一种模式”。修复是加一个值，同时补了一个遍历所有模式的回归测试。

7 还没做完的，以及接下来做什么

7.1 明确没做的

以下三项不是”还差一点”，是本次决定不做，理由如下。

**** 陪练角色的情绪实现。**** 设计文档里有完整方案（情绪状态、语气档位、触发条件、禁止句式），代码里没有。题目允许非主角角色只做设计，我把时间投给了军师和老师。需要强调的是：如果问”陪练有没有情绪”，准确答案是设计上有、实现上没有。

**** 联网 PVP。**** 本地对战是同一台设备上分屏轮流操作，服务端仍然接受客户端传来的状态快照，不是权威对局服务。真实竞技要求服务端下发可见状态和模式，否则改一个字段就能绕过限制。这条差距不是工程量问题，是架构问题。

****LLM 权重训练。**** 本机是 `arm64`，没有可用的 `CUDA` 后端，也没有云上 GPU 预算。仓库里记录的 `cloud_budget` 是 `not supplied`。

7.2 受条件限制没做的

需要外部条件才能完成的，与上面区分开。

项目	缺什么
独立大样本模型质量评测	独立标注集与 API 预算；现有 6 条真实样本不构成质量结论
检索显著性	需要把查询集扩到百条量级，否则差异无法与噪声区分
无提示学习迁移	需要真人被试；机制已实现（独立成功与提示后成功分开记录），缺的是数据
慢模型与长局浏览器验收	需要可控的慢网环境与完整走查
语音听感	需要真人耳朵；本机无音频输出，我无法自验
两名真人多局对战	需要两个人；分屏逻辑我用脚本模拟点击验证过

7.3 接下来的动作

按可核查的方式写，每项都带完成标志。

最后一项需要说明：这台机器上没有可见桌面和音频，我生成的是逐步截图序列加文字说明（在 `output/demo/` 与 `docs/DEMO-WALKTHROUGH.md`），它不是录像。要让评审

动作	完成标志	预计
把评测查询集从 16 条扩到 100 条以上	能对关键词与混合方案给出显著性检验结果	半天
补一轮 30-50 条真实模型用例	产出工具选择正确率与多余调用率	半天
给输出校验补因果语义检查	已复现的”取消说成命中”用例转为通过	已完成
把服务端与客户端的模式白名单做成同源常量	不再出现只改一侧的情况	1 小时
删除或归档已停用的语音代码	仓库里不再有无法验证的能力声明	1 小时
录一段完整演示	五条链路各有一段可播放记录	半天

看到真实操作过程，需要人在正常桌面环境下录一次。

8 复现

```

npm start                #                127.0.0.1:8765
npm test                  # 131
npm run build:knowledge   #
npm run eval:retrieval    #
npm run train:intervention #

```

外部来源与它们各自的使用边界记在 `knowledge/sources.json`、`docs/DOMAIN-RESEARCH.md`。官方文档、公开转载与我的推断分开标注。